AAS Time Off Planner: Technical Requirements Document

Version 1.4

23 October 2026

1. Document control

Item	Detail
Document	Technical Requirements Document (TRD)
Version	1.4 (see change log, section 22)
Date	23 October 2026
Companion	Product Requirements Document (PRD) v1.4; requirement IDs FR- and NFR- refer to it
Code	`github.com/SeanPodnar/AAS_Time_Off_Planner_V1`, folder `app/`
Built so far	Migrations 001 to 011: companies, people, audit log, company data separation, email sign-in, sessions, roles, platform owner sign-in, two-step sign-in, people management, company settings, bulk import of people, time off plans, accrual ledger, statutory reference rules, balances, requests and approvals, reminders and backup approvers, public holidays, team calendar and calendar feeds, reports and exports. 191 automated tests.

2. Architecture

```
Browser (employee, manager, HR admin, platform owner)
  | HTTPS: name.<platform domain> or a custom domain
  v
Load balancer: TLS, sets X-Forwarded-Host/For
  |                                  [AWS Canada Central]
  v
Next.js app (Node 22, TypeScript)
  |  server components and server actions
  |  background jobs: accruals, year end, reminders,
  |  payroll files, calendar push, webhooks
  |  two database roles, never superuser
  v
PostgreSQL 16, row-level security on company tables
  |
  +-- backups and point-in-time recovery,
      copies in AWS Canada West
```

```
Email: Amazon SES (Canada)    Files: S3 (Canada)
Secrets: AWS Secrets Manager
```

One application serves every company. The request's host decides the company. The database, not the application code, enforces separation between companies.

3. Technology stack

Layer	Choice	Version (pinned)	Reason
Runtime	Node.js	22 LTS	Long-term support
Web framework	Next.js (App Router), React	15.5.28, 19.1.10	Server rendering, server actions with built-in origin checks
Language	TypeScript, strict	5.9.3	
Database	PostgreSQL	16	Row-level security, JSONB, mature managed options in Canada
Database driver	postgres.js, plain SQL migrations	3.4.9	Separation rules readable as SQL; no ORM layer between app and policies
Validation	Zod	4.6.7	Upgraded for advisory GHSA-mrhx-c7xc-m3w5
QR codes	qrcode	1.5.4	Two-step set-up
Tests	Vitest	5.0.5	Unit and real-database tests
CI	GitHub Actions with a Postgres 16 service		Audit, lint, type check, tests, build on every pull request

Dependency policy: exact versions, `npm audit --audit-level=high` blocks merges, PostCSS forced to a patched version through `overrides` while on Next.js 15.

4. Company data separation (NFR-01)

4.1 Database roles

Role	Can	Cannot
Admin (owner)	Run migrations only	Is never used by the running app
`aas_app`	Read and write company tables, only for the company set on the current transaction; look up one company's public branding by host	Read the companies table; read platform tables; bypass row-level security
`aas_platform`	Manage companies and platform owner sign-in tables	Read any company table (people, sessions, audit log and so on)

4.2 Row-level security

- Every company table has `tenant_id`, row-level security **enabled and forced**, and a policy `tenant_id = app_tenant_id()` for both reading and writing.

- `app_tenant_id()` reads the transaction setting `app.tenant_id`. With nothing set it returns NULL, so queries return no rows (fails closed).
- The app reaches company data only through `withTenant(id, fn)`, which opens a transaction and sets the company with `set_config(..., true)`, so the setting ends with the transaction and cannot leak to the next request on a pooled connection.
- Links between rows use composite foreign keys `(tenant_id, id)`, so a manager, session or factor can never point at another company's person, even for the admin role.
- A guard test lists every table with a `tenant_id` column and fails if any lacks forced row-level security and an `aas_app` policy.

4.3 Moving a company to its own database later

UUID keys everywhere (no shared sequences), no cross-company foreign keys, files under a per-company prefix. A tenant registry will map company to connection string; moving a company is copy, verify counts and checksums, switch the registry entry, delete from the shared database.

5. Data model

5.1 Built (migrations 001 to 011)

Table	Purpose	Key columns
`tenants`	Companies	legal_name, short_label (unique ignoring case and spaces, 24 max), subdomain, custom_domain, brand_color, country, status
`people`	Employees	tenant_id, full_name, email (unique per company), job_title, employee_number (unique per company), jurisdiction (`CA-ON`, `US-CA`), hire_date, manager_id (same company, not self), team_id, location_id, role, status, hours_per_day, work_days (ISO weekdays, part-time)
`tenant_settings`	Company settings	time_zone (IANA), second_approval_threshold (default 10). Defaults apply until first saved.
`teams`, `locations`	Grouping	Name unique per company ignoring case; created by typing a new name
`person_managers`	Secondary managers	See and are notified; only the primary manager approves
`import_batches`	Bulk import staging	Parsed file rows (JSON), created_by (must be an HR admin of the same company), expires

Table	Purpose	Key columns
		after one hour, used once, under row-level security
audit_log	Append-only history	tenant_id, at, actor, action, detail (JSON). Update and delete blocked by trigger and grants.
login_tokens, sessions	Email sign-in	SHA-256 token hashes, expiry, used or revoked time, method, mfa_at
mfa_factors, mfa_recovery_codes	Two-step sign-in	Encrypted key, confirmed time, last used step; hashed recovery codes
platform_owners, platform_login_tokens, platform_sessions, platform_mfa_factors, platform_mfa_recovery_codes	Platform owner sign-in	Reachable only by aas_platform
plans	Time off plans	category, unit (days or hours), accrual method and timing, plan year start, proration and rounding, carryover rule, cap, expiry, maximum balance, approval and privacy settings. Name unique per company ignoring case
plan_levels	Levels per plan	annual amount in the plan's unit, tenure tiers as JSON [{years, amount}]
plan_assignments	One level per person per plan at a time	effective_from, effective_to (exclusive, null for open)
accrual_ledger	Every balance movement	effective_date, plan_year, kind, amount, unique source_key per company, reason, rule_ref. Append-only: the app role has select and insert only
jurisdiction_rules	Statutory reference data, not company data (no tenant_id)	jurisdiction, rule kind (min_entitlement, no_forfeit, forfeit_conditional), years of service, amount in weeks, effective_from, verified flag
requests	One ask: new time off, a change to approved time off, or a cancellation	kind, target_id, start and end date with part (full, am, pm), days, amount in the plan unit, status, requester_time_zone, admin_reason

Table	Purpose	Key columns
request_days	Each counted day of a request	day, part (full, am, pm)
approval_steps	Each approval step	step_no, role (manager, next_manager, admin), approver_id, status, override, pending_since, reminded_at, escalated_at
approval_backups	Backup approver for a date range	approver_id, backup_id, start_date, end_date
company_holidays	Company changes to the default holidays	day, action (add or skip), name, jurisdiction (null for everyone), reason
calendar_feeds	Private calendar feed links	scope (me, team, reports), SHA-256 token hash, revoked_at, last_used_at

tenant_settings gained reminder_after_days and escalate_after_days (migration 008); plans gained cancel_needs_approval (migration 007). Migration 011 adds current_tenant_legal_name(), which returns only the current company's legal name for reports.

5.2 Planned for v1

Table	Purpose
tenant_settings (more columns)	Retention, sign-in methods allowed, allowed email domains
payroll_exports, payroll_codes	Pay period files and plan to WorkLinks code mapping
notifications_outbox	Emails queued in the same transaction as the change that caused them
jobs	Background job queue (Postgres SKIP LOCKED)

6. Request routing and branding

- parseHost(host, rootDomain): the bare root is the platform console; one subdomain level is a company; any other host is looked up as a custom domain. Look-alike domains (evilaas-timeoff.ca) are not treated as subdomains.
- The company is resolved through the security-definer function resolve_tenant_by_host, which returns only id, short label, brand colour and country for non-suspended companies.
- The host is read from X-Forwarded-Host first. Next.js renders a server action's redirect with Host set to its own address, and in production the load balancer sets the header. A forged value only selects which company's records are checked; it must match a company, and sessions are looked up inside that company, so it cannot reach another company's data. The load balancer must overwrite, not append, client-supplied values.
- Brand colour drives CSS variables; button text colour is chosen by contrast (WCAG formula).

7. Sign-in and sessions

7.1 Email link (built)

1. The person enters an email on their company's address. Rate limits: 5 per address, 30 per network address per 15 minutes.
2. If the address belongs to an active person in that company, a 256-bit random token is stored as a SHA-256 hash with a 15-minute expiry and emailed. The answer is identical either way.
3. The link opens a Continue page; the POST redeems the token once and creates a session.

7.2 Sessions (built)

256-bit token, stored hashed, 12-hour expiry, cookie `HttpOnly`, `SameSite=Lax`, `Secure` in production, host-only (no `Domain` attribute) so a session never travels to another company's address. Sign-out revokes the row. Server actions get Next.js's origin check against cross-site requests.

7.3 Two-step sign-in (built)

RFC 6238 codes (SHA-1, 6 digits, 30 seconds, one step of drift either way), verified against the RFC test vectors. The key is encrypted with AES-256-GCM using `MFA_ENCRYPTION_KEY` with the person's id as associated data. The last used step is stored so a code works once; recovery codes are hashed and single use; 5 wrong codes per session lock it for 15 minutes. Required for HR admins and the platform owner; checked on every protected page.

7.4 Google and Microsoft 365 (next)

- OpenID Connect authorization code flow with PKCE, state and nonce.
- Google rejects wildcard return addresses, so all companies share one fixed return address on the root domain: `https://<root>/auth/<provider>/callback`. The start request carries the company in signed state. After the ID token is verified, the app creates a 60-second single-use hand-off code and redirects to the company's own address, which exchanges it for a session cookie on that host.
- Microsoft uses the multi-tenant (`organizations`) endpoint; the issuer is checked against the token's tenant id. Optionally each company can restrict sign-in to its own Google Workspace domain or Microsoft tenant id.
- The email in the token must be verified and must match an active person in that company. No accounts are created from sign-in.
- Secrets: `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `MICROSOFT_CLIENT_ID`, `MICROSOFT_CLIENT_SECRET` in the secret store. Microsoft secrets expire within 24 months; a reminder job warns 30 days ahead.

8. Accrual engine (FR-30 to FR-43)

- Built in `app/src/lib/accrual/`: `dates.ts` (calendar dates), `rules.ts` (statutory rules), `schedule.ts` (pure engine with no database access), `post.ts` (loading, posting and balances).
- Ledger-based: every balance movement is a dated row in `accrual_ledger`. A balance on a date is the sum of rows on or before it. Each row has a deterministic `source_key`, and posting uses `on conflict do nothing`, so reruns never double-credit.

- Earning methods: lump sum (plan year start, hire date or work anniversary), periodic (annual, semi-annual, quarterly or monthly, credited at the start or end of each period), and hours worked (salaried staff on their standard schedule, credited at each month end).
- Proration: months remaining, counting the start month when the start is on or before the 15th, rounded per plan to the half day (days) or quarter hour (hours). A start that still counts every month of a period gets the regular amount. Level and tenure changes mid-year add the prorated difference.
- Maximum balance: a credit is reduced to the room left under the cap, with the reason recorded.
- Year end: once a new plan year starts, the plan rule (none, capped, unlimited or maximum balance) writes a forfeit row; then jurisdiction rules effective that day write override rows naming the rule. US no-forfeit states keep everything. Canadian provinces keep up to the statutory minimum (weeks converted to days by the person's work days per week), counting what the plan already carries. Notice-required states (IL, MA) produce a warning, not a block.
- Carried days expire on the plan's expiry date if not used; bookings use carried days first. Expiry is blocked in no-forfeit states and limited to amounts above the statutory minimum in Canada.
- Posting today runs when a balance is viewed, for that person, using the company's own date (`todayIn` with the company time zone). A nightly job per company comes with requests and approvals.
- Balances return the amount available today, leave booked ahead, the projected balance at plan year end and, for HR admins, compliance notes (below the statutory minimum, next minimum step, figures pending legal review).
- Statutory figures in migration 006 come from secondary sources and are marked `verified = false` until the legal review (PRD D7).
- Working day count for requests: dates between first and last day, minus non-working days from the person's schedule, minus holidays for the person's jurisdiction and company, with half days on the first and last day (built with requests).

9. Approval workflow (FR-50 to FR-57)

- Built in `app/src/lib/requests/`: `days.ts` (pure day counting and the run of joined-up time off), `service.ts` (preview, submit, amend, cancel, decide, HR entry, balance adjustments, lists), `followups.ts` (reminders, escalation, backup approvers).
- The run counts the working days of the request plus every live booking joined to it (any plan, approved or pending), where weekends and holidays don't break it. If it exceeds the company threshold and the plan is manager-approved with a second step, steps are the primary manager then that manager's manager, falling back to HR. Automatic plans approve at once; HR-approved plans have one HR step.
- Steps are rows in `approval_steps` (waiting, pending, approved, declined, cancelled). The current step's approver, or their active backup, decides; an HR admin can decide any step, logged as an override. Nobody decides their own request. Decline needs a comment.
- Approval writes a booking row to `accrual_ledger` for each plan year the request touches; a change or cancellation writes a booking_reversal. A change to approved time keeps the original booked until the change is approved. Pending requests count against what is available, and writes for one person are serialised with a row lock so two requests can't both pass the balance check.
- Each decision (steps, request status, ledger rows, audit entry) is one transaction; emails go out after it commits.
- The scheduled job (`npm run jobs`, `scripts/nightly.ts`) posts accruals and then sends follow-ups for every company: one reminder after `reminder_after_days` business days and

one HR escalation after `escalate_after_days`, each at most once per step. It is safe to run hourly.

10. Calendars

- Built in `app/src/lib/calendar/`: `visibility.ts` holds the rules. The person, HR and managers (any level up, and secondary managers) see plan names and pending time; coworkers see approved time only, as the plan allows (plan name, "Out of office", or nothing).
- Team calendar (`/calendar`): a week grid of 2 to 6 weeks with filters; a plan filter never reveals a plan the viewer can't see.
- ICS feeds (`/feeds/<token>.ics`): scopes me, team and reports. Feeds never show more than a coworker sees, never show a private plan's name (even in the person's own feed) and never include comments. Tokens are 256-bit random values stored as SHA-256 hashes; a new link revokes the old one; unknown, revoked, other-company and inactive-owner links all return 404. Responses are `Cache-Control: private, no-store`.
- Direct push (v1.1): per-person OAuth to Google Calendar API and Microsoft Graph; events written from the outbox within seconds; tokens encrypted like two-step keys.

10.1 Time zones

- All timestamps are stored in UTC (`timestamptz`). Leave dates are calendar dates (`date`), never timestamps, so a day off is the same day for everyone; ICS events are all-day `VALUE=DATE` events and do not shift between zones.
- v1: one company time zone (`tenant_settings.time_zone`) decides "today", deadlines, reminders and year end. Each request stores `requester_time_zone`.
- v1.1 (FR-64): `people.time_zone`, nullable, falls back to the company zone. Rules that depend on "today" (cannot book the past, approval deadlines, escalation timers) use the requester's zone. Display converts timestamps to the viewer's zone and labels the requester's zone when it differs. Reminders and digests are scheduled per recipient at a local hour (default 09:00). Calendar push writes all-day events, plus the person's own zone for any timed reminders. Report run labels state the zone used.

11. Integrations

Integration	Design
WorkLinks (v1)	Pay period file per company in WorkLinks' import layout (to be confirmed, D5). Approved absences by payroll code; corrections for periods already sent are separate rows. Download or SFTP.
WorkLinks API (blocked)	Same mapping and outbox; transport swapped once documentation exists.
Jira (v1.1)	Per-company OAuth 2.0 (3LO) to the company's Jira Cloud site, least-privilege scopes. Absence issue per approved absence with dates and original estimate; updated or cancelled from the outbox.
Availability API (v1.1)	`GET /api/v1/availability?email&from&to` returns per-day out or available with half days. Per-company API keys (hashed, scoped, rotatable, rate limited, optional IP allow list). Webhooks signed with HMAC-SHA256 and a timestamp.

Integration	Design
MCP server (backlog)	Tools for balances, requests, who's out, availability, request and approve; OAuth per person; same authorisation and privacy code as the web app.

12. Reports and exports

- Built in `app/src/lib/reports/`: one `Report` shape (title, company legal name, run label, filters in words, typed columns with units, rows, totals, notes) drives the screen, the Excel file and the CSV.
- Balance reports (balance summary, liability in hours, year-end preview, compliance) use the accrual engine, so they match the balances on screen. Activity reports (who's out, absence rate, approval metrics, audit) query requests, approval steps and the audit log.
- Excel files are written directly as Office Open XML and zipped with `fflate`: run label and filters first, header row frozen, auto-filter on, real dates and percents. CSV is UTF-8 with a byte-order mark; text starting with = + - @ is prefixed so it can't run as a formula.
- HR admin only, every query inside `withTenant`; every export writes an audit entry with its filters; downloads are `Cache-Control: private, no-store`.

13. Email

- Development: emails print in the terminal and go to an in-memory outbox for tests.
- Production: Amazon SES in Canada (Central), sending domain with SPF, DKIM and DMARC. Emails are queued in `notifications_outbox` in the same transaction as the change and sent by a job with retries.
- Subjects start with the company short label. Bodies avoid private plan names except to the person, their approvers and HR.

14. Security controls (NFR-04)

Control	Status
Separation by database roles and forced row-level security, with guard test	Built
Tokens and recovery codes stored as SHA-256 hashes; two-step keys AES-256-GCM	Built
Host-only HttpOnly cookies; origin-checked server actions	Built
Rate limits on sign-in links and codes	Built (in memory; see 19)
Security headers: nosniff, frame deny, strict referrer, permissions policy	Built
Content Security Policy with nonces	Planned v1
Dependency audit on every pull request	Built
Secrets in AWS Secrets Manager; no secrets in the repository	Planned for release (development uses throwaway values)

Control	Status
Encryption at rest (RDS, S3) and TLS 1.2+ everywhere	Planned for release
Audit log append-only, including platform owner visits	Built (visits pending)
Calendar feed tokens stored as SHA-256 hashes, revocable, rate limited	Built
Spreadsheet formula injection guard on CSV and Excel exports	Built
Server actions act only on an explicit button choice (no default to approve or replace)	Built
Penetration test and review against OWASP ASVS level 2 before real data	Planned (board item)

15. Hosting, environments and cost

Phase	Where	Data
Build (now)	Local and CI; optional free Supabase project in Canada (Central)	Synthetic only
Release	AWS Canada (Central), Montreal: app on ECS Fargate or App Runner, RDS PostgreSQL 16 with point-in-time recovery, SES, S3, Secrets Manager; backup copies in Canada West, Calgary	Real

Estimated release cost for 4 companies and under 100 people: about US$60 to US$110 a month (smallest RDS instance with backups about $20 to $30, one small app container $15 to $30, load balancer about $20, the rest a few dollars). Confidence: moderate; to be priced with the AWS calculator at release.

Environments: development (from `main`, synthetic data), production (from release tags, after the security review).

16. Delivery process

- Trunk-based: `main` always green; one branch per board item; pull request with the template; CI must pass; owner approves.
- Releases tagged `v1.0.0`, `v1.1.0`; hotfixes `v1.0.x`.
- GitHub branch protection is configured but not enforced on the free plan for a private repository; GitHub Pro (about US$4 a month) enforces it.
- The PRD and TRD are updated after each merged feature, in a small documentation pull request, with a change log entry.
- Model delegation: simpler, low-complexity tasks (documentation, status updates, standard screens) go to lower AI models and are reviewed by the orchestrating session before a pull request; the owner is asked before escalating to a higher model for complex or high-risk work (accrual and statutory logic, approvals, sign-in, data separation). See `CLAUDE.md`.

17. Testing strategy

Level	What	Where
Unit	Validation, host parsing, contrast, tokens, rate limits, TOTP (RFC vectors), encryption, accrual and day-count functions	Every PR
Database	Separation tests (cannot read, insert, move, update, delete across companies; no cross-company links; no leak across pooled connections; role limits; guard test), sign-in and two-step flows against Postgres 16	Every PR
End to end	Browser runs of each role's main flows (Playwright)	Each feature PR; automated in CI in v1
Accrual golden tests	Worked examples for proration, periodic credits, tenure, maximum balance, year end, overrides and expiry (20 cases built); province and state examples to be checked by the legal reviewer	Every PR; legal check before release
Security	Dependency audit every PR; penetration test before release	

18. Observability and operations

Structured logs without personal data beyond ids; error tracking hosted in Canada or self-hosted; health check endpoint; alerts on job failures, email bounces and sign-in spikes; daily backup restore test in the first month, monthly after.

19. Known limitations and technical debt

Item	Plan
Rate limits are in memory (one server)	Move to Postgres when running more than one instance
Production email transport not wired	SES with the release item
Next.js 15 with PostCSS override	Move to Next.js 16 when stable for our needs; remove override
`X-Forwarded-Host` trusted	Load balancer must overwrite the header; documented in deployment checklist
Branch protection not enforced	GitHub Pro
Two-step not offered to employees	Decision D8
Scheduled jobs run from a script; nothing schedules them yet	Run `npm run jobs` hourly from the host's scheduler at release
Changing a level's amount or a plan's carryover rule applies from the next posting only; rows already posted are not recalculated	Admin adjustment screen; recalculation preview later

Item	Plan
Inactive people are skipped by posting	Revisit with offboarding and payout reports
Canadian statutory protection counts carried days toward the minimum (the reading less generous to employees)	Confirm with the legal reviewer (PRD D7)
Feed rate limit is in memory	Move with the other rate limits
A pending change to approved time is not shown on the calendar (the approved original is)	Show pending changes to managers later
Changing company holidays doesn't recount requests already booked	Recount tool for HR later
Audit report shows at most 5000 rows per run	Narrow the date range or export in parts

20. Open technical decisions

#	Decision	Recommendation
T1	App hosting on AWS: ECS Fargate or App Runner	App Runner for simplicity if available in Canada (Central) at release; otherwise Fargate
T2	~~Background jobs: Postgres queue or a managed queue~~	Resolved for v1: an idempotent scheduled script run hourly; a queue only if jobs grow
T3	Error tracking with Canadian residency	Self-hosted or a vendor with a Canadian region
T4	~~Work schedules for part-time staff~~	Resolved: built as `people.work_days` and `people.hours_per_day`
T5	~~Company time zone~~	Resolved: built as `tenant_settings.time_zone`; per-person zone in v1.1 (section 10.1)

21. Traceability

Requirement	Section
FR-01 to FR-06	4, 5, 6
FR-10 to FR-15	7
FR-20 to FR-22	5.2
FR-30 to FR-43	5.1, 8 (FR-41: `app/src/lib/holidays/`)
FR-50 to FR-57	5.1, 9
FR-60 to FR-64	5.1, 10, 10.1
FR-70 to FR-80	12
FR-90 to FR-95	11
FR-100, FR-101	9, 13
FR-110	4, 14

Requirement	Section
NFR-01 to NFR-11	4, 14, 15, 17, 18

22. Change log

Version	Date	Changes
1.0	23 Oct 2026	First full scope.
1.1	23 Oct 2026	Migration 004 built (company settings, teams, locations, secondary managers, employee number, work days and hours per day). Added section 10.1 on time zones and `requester_time_zone` on requests. T4 and T5 resolved. Documentation updated after each merged feature.
1.2	23 Oct 2026	Migration 005 (import_batches) built for bulk import of people. Model delegation rule added to the delivery process.
1.3	23 Oct 2026	Migration 006 built: plans, levels, assignments, append-only accrual ledger, statutory reference rules. Section 8 rewritten for the built engine. Testing, known limitations and traceability updated. 124 automated tests.
1.4	24 Oct 2026	Migrations 007 to 011 built: requests, request days, approval steps, reminder settings and backups, company holidays, calendar feeds, report company name. Sections 9, 10 and 12 rewritten for what is built. Security, limitations, T2 and traceability updated. 191 automated tests.